

Pokedex

Defensa Técnica (Next.js + T3 Stack)

- Autor: Marcos Barranquero
- Enfoque: arquitectura limpia, rendimiento y UX
- Monorepo preparado para despliegue real

1. Qué he construido (en claro)

Una Pokedex moderna con estética inspirada en consola, navegación fluida por colección, vista de detalle rica en información y búsqueda con soporte de evoluciones.

La app mantiene contexto de usuario entre navegación, soporta multiidioma y personalización visual, e integra generación opcional de fun facts con IA a través del BFF.

Se ha implementado con foco de producto real: arquitectura sólida, buena respuesta, pruebas automáticas, Docker y despliegue en cloud.

2. Funcionalidades destacadas

UX principal

- Listado ordenado por ID
- Filtros por generación y multitype
- Búsqueda con evoluciones
- Vista detalle inline con stats y evoluciones

Valor adicional

- i18n (EN, ES, IT, PT, DE)
- Persistencia de color de carcasa GBA
- Responsive real (desktop/tablet/móvil)
- Fun fact IA con botón de regeneración

3. Arquitectura en una diapositiva

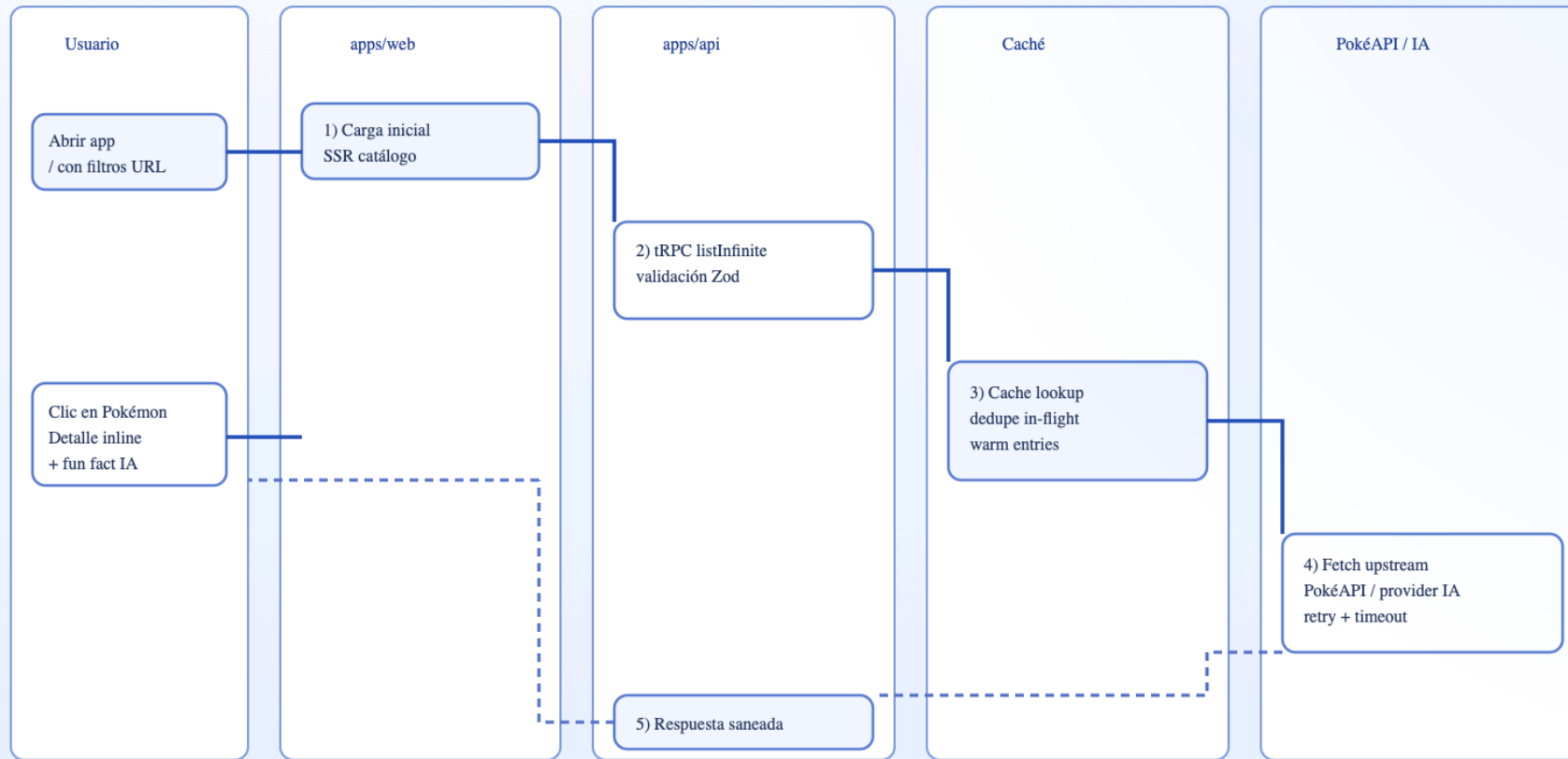
Arquitectura del monorepo (BFF-first)

Pokedex · Next.js + tRPC + Fastify + Shared Contracts



4. Flujo de llamadas (camino feliz)

Flujo de llamadas: catálogo y detalle



5. Por qué estas decisiones técnicas

tRPC frente a REST ad-hoc

- Front y back comparten el mismo contrato
- Menos errores de integración entre interfaz y backend
- Más velocidad para evolucionar funcionalidades

Fastify frente a Express

- Muy buen rendimiento para este tipo de API
- Configuración clara y mantenible
- Encaja muy bien con la capa BFF

Zustand frente a Redux

- Gestión de estado simple y directa
- Menos código para mantener
- Ideal para filtros y estado de interfaz

6. Trade-offs asumidos (conscientes)

- **Filtros y búsqueda en URL:** permite compartir enlaces con el estado exacto, a cambio de algo más de lógica.
- **Renderizado inicial en servidor:** mejora la primera carga y el posicionamiento.
- **Caché en backend:** reduce esperas y evita repetir llamadas costosas.
- **IA opcional:** la app funciona bien con o sin IA.

7. Estrategia de rendimiento

Red y caché

- Caché en backend para acelerar respuestas
- Política de reintentos ante fallos puntuales
- Calentamiento inicial para reducir tiempos de espera

Renderizado

- Primera pantalla preparada en servidor
- Estado de filtros sincronizado con la URL
- Carga progresiva al hacer scroll

8. Modelado de datos y contratos

- Reglas de datos compartidas entre frontend y backend
- Contrato único para entradas y salidas de la API
- Validaciones para evitar errores en tiempo de ejecución

Resultado: cambios más seguros y menos incidencias por desajustes de datos.

9. Integración de IA

Estrategia de proveedores:

- `none` -> fun fact determinista
- `ollama` -> modelo local
- `groq` -> modelo hosted de baja latencia

Prompt único compartido para mantener consistencia entre proveedores.

Frontend simple: solo solicita/regenera el fun fact; la elección del proveedor se decide en backend.

10. Decisiones UI/UX

- Carcasa estilo GBA para identidad memorable
- Estados visuales claros: loading, empty, error
- Interacción por teclado y ratón en filtros
- Matrices responsive de colección:
 - Desktop: 2x6
 - Tablet: 2x3
 - Móvil: 2x2

11. Calidad y entrega

Calidad automática

- Unit tests (API + Web)
- Contract tests
- E2E (Playwright)
- Typecheck y pipeline CI

Calidad de entrega

- Dockerfiles multietapa
- Perfiles Docker Compose
- Despliegue validado en Railway
- Historial de releases con changelog

12. Herramientas de IA en el proyecto

GitHub Codex

- Evaluación de alternativas de arquitectura
- Implementación iterativa de funcionalidades
- Refactorización y mejora continua del código

Flujo asistido con IA

- Revisión de PRs con GitHub Copilot
- Ciclo de releases con validación continua
- Uso de skills de agentes para tareas específicas

Gracias

Preguntas